

Software Distribuït

PRÀCTICA I: MEMÒRIA FINAL

Mario VILAR

20392724

David DíEZ

20392643

1 d'abril de 2025

Professor: Kaisar Kushibar



UNIVERSITAT DE
BARCELONA

ÍNDEX

1 Informació general

2 ComUtils

2.1	BattleshipComUtils	2
2.2	BattleshipGame	5
2.3	GameBoard	8
2.4	Messages	10

3 Server

3.1	GameHandler	11
3.2	Server	13

4 Client

4.1	GameClient	13
-----	----------------------	----

5 Altres qüestions

5.1	Multiplayer	14
5.2	Problemes	15

1 INFORMACIÓ GENERAL

El projecte desenvolupat en aquesta pràctica consisteix en la implementació del joc Battleship seguint una arquitectura client-servidor i una especificació detallada del protocol de comunicació.

La pràctica ha estat dividida en diferents components lògics, cadascun amb responsabilitats clares dins del sistema. S'han implementat diverses classes per gestionar la comunicació entre client i servidor (`BattleshipComUtils`), la lògica de joc (`BattleshipGame`, `GameBoard`), la gestió dels missatges segons el protocol (`Message` i subclasses) i el control de l'execució (`GameHandler`, `GameClient`). També s'ha afegit una extensió per donar suport al mode multijugador real amb l'objectiu de permetre partides entre diferents terminals i clients connectats.

2 COMUTILS

2.1 BATTLESHPCOMUTILS

1. Descripció general

- La classe `BattleshipComUtils` estén la classe `ComUtils` per proporcionar funcionalitats específiques per al joc Battleship.
- Gestió de la lectura i escriptura de missatges segons el protocol, tant per a enviar com per a rebre diferents tipus de missatges.
- Proporciona mètodes per tancar els streams de manera segura.

2. Mètodes de escriptura de missatges

- `writeRejoinMessage(RejoinMessage message)`
 - Escriu un missatge de rejoin a l'output stream.
 - Primer escriu el codi del missatge (`MessageType.REJOIN`), i després el nom del jugador amb un màxim de 50 caràcters.
- `writeGetConfigMessage(GetConfigMessage message)`
 - Escriu un missatge per obtenir la configuració.
 - Primer escriu el codi de missatge (`MessageType.GETCONFIG`), després l'identificador del jugador i de la partida en format enter de 32 bits.
- `writeGameConfigMessage(GameConfigMessage message)`
 - Escriu un missatge de configuració del joc.
 - Primer el tipus de missatge (`MessageType.GAMECONFIG`), després la mida del tauler i la configuració d'embarcacions.

- `writeAddVesselMessage(AddVesselMessage message)`
 - Escriu un missatge d'afegir vaixell.
 - Escriu primer el tipus (`MessageType.ADDVESSEL`), després la id del jugador, id de la partida, el tipus de vaixell i finalment les posicions inicial i final.
- `writeGetStatusMessage(GetStatusMessage message)`
 - Escriu un missatge d'estat.
 - Primer el tipus (`MessageType.GETSTATUS`), i després la id del jugador i id de la partida en format enter de 32 bits.
- `writeGamestatusMessage(GameStatusMessage message)`
 - Escriu un missatge d'estat de joc.
 - Escriu primer el tipus (`MessageType.GAMESTATUS`), l'estat del joc, la mida del tauler i els dos taulers.
 - Segons l'estat del joc, escriu un nombre variable de bytes addicionals (info).
- `writeShotMessage(ShotMessage message)`
 - Escriu un missatge de tret.
 - Escriu primer el tipus (`MessageType.SHOT`), i després la id del jugador, de la partida i la posició del tret.
- `writeHitMessage(HitMessage message)`
 - Escriu un missatge de cop encertat.
 - Escriu el tipus de missatge (`MessageType.HIT`) i si s'ha enfonsat (sink).
- `writeFailMessage(FailMessage message)`
 - Escriu un missatge de cop fallit.
 - Només escriu el tipus de missatge (`MessageType.FAIL`).
- `writeLeaveMessage(LeaveMessage message)`
 - Escriu un missatge de sortida.
 - Escriu el tipus (`MessageType.LEAVE`), seguit de la id del jugador i la id de la partida.

3. Mètodes de lectura de missatges

- `readMessage()`
 - Determina el tipus de missatge i crida el mètode corresponent per parsejar els seus camps.
 - Retorna un missatge des de l'input stream.
- `readErrorMessage()`
 - Llegeix el tipus d'error i una descripció.
 - Retorna el missatge d'error.

- `readCreateMessage()`
 - Llegeix el nom del jugador, dimensions del tauler, vaixells (5 bytes) i si és AI.
 - Retorna un missatge de creació.
- `readOkMessage()`
 - Llegeix la id del jugador i la id de la partida.
 - Retorna un missatge d'OK.
- `readJoinMessage()`
 - Llegeix el nom del jugador.
 - Retorna un missatge de join.
- `readRejoinMessage()`: retorna un missatge de rejoin i llegeix el nom del jugador.
- `readGetConfigMessage()`: retorna un missatge per obtenir la configuració i llegeix la id del jugador i la id de la partida.
- `readGameConfigMessage()`
 - Llegeix les dimensions del tauler (width i height) i els vaixells.
 - Retorna un missatge de configuració del joc.
- `readAddVesselMessage()`
 - Llegeix la id del jugador, id de la partida, tipus de vaixell, posició inicial i final.
 - Retorna un missatge d'afecció de vaixell.
- `readGameStatusMessage()`
 - Llegeix l'estat del joc (byte), la mida del tauler, els dos taulers i, segons l'estat, un nombre variable de bytes addicionals.
 - Retorna un missatge d'estat de joc.
- `readShotMessage()`: Llegeix un missatge de tret i llegeix la id del jugador, id de la partida i la posició del tret.
- `readHitMessage()`
 - Llegeix un byte que indica si s'ha enfonsat el vaixell.
 - Retorna un missatge de cop encertat.
- `readFailMessage()`
 - No requereix lectura addicional, ja que només s'identifica pel tipus.
 - Retorna un missatge de cop fallit.
- `readLeaveMessage()`: Llegeix un missatge de sortida i llegeix la id del jugador i la id de la partida.
- `readGetStatusMessage()`: Llegeix un missatge d'estat i llegeix la id del jugador i la id de la

partida.

4. Altres mètodes:

- `closeStreams()`:
 - Tanca de manera segura els streams d'entrada i sortida.
 - Abans de tancar, fa un flush del `dataOutputStream`.
 - Captura les excepcions d'I/O i imprimeix un missatge d'error si cal.

2.2

BATTLESHIPGAME

La classe `BattleshipGame` representa una partida de Battleship on els jugadors poden afegir vaixells, disparar. L'estat del joc és controlat pel `GameHandler`, tot i que es podria argumentar que en funció de la sortida d'alguns mètodes (per exemple, l'`addVessels` pot retornar *true* o *false* en funció de si s'ha pogut afegir el vaixell correctament) el mateix *BattleshipGame* en fa una gestió implícita. Aquesta classe:

- Implementa la interfície `IBattleshipGame`.
- Utilitza la classe `BattleshipComUtils` per enviar missatges d'estat del joc.
- Gestiona els jugadors, els seus taulers i els vaixells.

Podem distingir els mètodes segons la seva funcionalitat:

1. Constructor i inicialització

- `BattleshipGame(int gameId, BattleshipComUtils comutils)`:
 - Comprova que el `gameId` tingui exactament 5 dígit; en cas contrari, llença una `BattleshipException` amb `ErrorType.INVALID_GAME_ID`.
 - Assigna la instància de `comutils` i guarda el `gameId` per a la partida.

2. Gestió dels taulers (Boards)

- `createBoards(int playerId, byte W, byte H)`: Crea un tauler per al jugador si aquest no existeix prèviament, utilitzant les dimensions indicades.
- `getOwnBoard(int playerId)`: Retorna el tauler associat amb el jugador indicat.
- `getOpponentBoard(int playerId)`: Retorna el tauler de l'oponent a partir del jugador actual (`getOpponentId(playerId)`).
- `getOppBoard(int playerId)`:
 - Retorna una versió modificada del tauler de l'oponent per visualització.
 - Processa cada cel·la: si el valor és divisible per 10 (i diferent de 0), mostra el valor dividit per 10; en cas contrari, la cel·la es mostra amb 0. Això és així pel que demana el protocol i les instruccions de la pràctica.
- `getBoardDimensions()`: Retorna les dimensions (ample i alt) del tauler (o d'algun tauler si

n'hi ha més d'un); si cap tauler existeix, retorna null.

3. Gestió dels dispars (Shots)

- `shot(int playerId, int r, int c):`
 - Processa un dispar a una cel·la específica. Comprova que la fila (r) i la columna (c) estiguin dins dels límits del tauler.
 - En cas de límits invàlids, retorna -1; si són vàlids, delega el processament a `processShot` (convertint els índexs a o-indexed).
- `shot(int playerId):`
 - Processa un dispar aleatori seleccionant fila i columna amb la classe `Random` segons les dimensions del tauler.
 - Delegeix el processament en `processShot`.
- `processShot(int playerId, int r, int c):`
 - Verifica que el jugador existeix i que és l'actiu.
 - Processa un dispar sobre el tauler de l'oponent.
 - Segons el resultat del dispar (retornat per `board.shot(r, c)`), si és 2 (vaixell tocat), elimina una instància del vaixell amb `removeVessel`.

4. Gestió dels jugadors (Players)

- `addPlayer(int playerId, byte W, byte H, boolean ai):`
 - Afegeix un jugador al joc si no existeix prèviament i si no s'ha arribat al límit de 2 jugadors.
 - Registra si el jugador és controlat per IA.
 - Inicialitza els vaixells i crea el tauler utilitzant `createBoards`.
 - Si és el primer jugador, s'estableix com a jugador actiu.
- `playerExists(int playerId):` Retorna true si el jugador ja està registrat; false en cas contrari.
- `removePlayer(int playerId):` Elimina el jugador, el seu tauler i la seva configuració de vaixells dels mapes corresponents.
- `setActivePlayer(int playerId) i getActivePlayer():`
 - `setActivePlayer` assigna el jugador actiu.
 - `getActivePlayer` retorna l'identificador del jugador actiu, retornant -1 si el joc ha finalitzat (codi d'estat 4).
- `setWinPlayer(int playerId) i getWinPlayer():`
 - `setWinPlayer` registra el jugador guanyador.
 - `getWinPlayer` retorna l'identificador del guanyador, o -1 si el joc encara no ha finalitzat.

- `getNumPlayers()` i `getPlayerId(int idx)`:
 - `getNumPlayers` indica el nombre total de jugadors.
 - `getPlayerId` retorna l'identificador del jugador segons la posició en la col·lecció; si l'índex és erroni, retorna -1.
- `getOpponentId(int playerId)`: Retorna l'identificador del jugador contrincant, o -1 si no hi ha prou jugadors.
- `isPlayerReady(int playerId)` i `allPlayersReady()`:
 - `isPlayerReady` comprova si el jugador ha col·locat tots els vaixells.
 - `allPlayersReady` retorna true si tots els jugadors estan preparats.
- `isBot(int playerId)`: Retorna true si el jugador és controlat per IA.

5. Gestió dels vaixells (Vessels)

- `setVessels(byte[] vessels)` i `getVessels()`:
 - `setVessels` actualitza la configuració total de vaixells (ha de ser de 5 elements).
 - `getVessels` retorna aquesta configuració.
- `addRandomVessels(int playerId)`:
 - Per a jugadors IA, genera posicions aleatòries i afegeix els vaixells fent ús de `addVessel`.
- `addVessel(int playerId, int ri, int ci, int rf, int cf, int type)`:
 - Tenta afegir un vaixell al tauler del jugador, comprovant que no s'excedeixi el nombre total permès per aquell tipus.
 - Si es col·loca correctament, incrementa el compte de vaixells per aquell tipus.
- `getRemainingVessels(int playerId, int type)`:
 - Retorna quants vaixells queden per al tipus indicat. S'utilitza per veure si ja estan tots els vaixells ficats o enfonsats.
- `allVesselsSunk(int playerId)`:
 - Comprova si per a cada tipus de vaixell no en queda cap per a un jugador; retorna true si tots estan enfonsats.
- `removeVessel(int playerId, int type)`:
 - Disminueix el compte de vaixells d'un tipus determinat en cas que el vaixell sigui tocat o enfonsat.

6. Gestió de l'estat del joc (State)

- `setGameState(GameState state)`:
 - Actualitza l'estat global del joc (per exemple, `WAITING_PLAYERS`, `PLAYING`, `FINISHED`, etc.).

- `notifyStatusToEveryone()` i `notifyStatus(int playerId)`:
 - `notifyStatusToEveryone` iterarà sobre tots els jugadors, cridant `notifyStatus` per cadascun.
 - `notifyStatus` envia un missatge d'estat (`GameStatusMessage`) al jugador, que inclou:
 - L'estat del joc.
 - Les dimensions i contingut del seu tauler.
 - Una versió modificada del tauler de l'oponent.
 - Informació addicional obtinguda amb `computeInfo`.
- `computeInfo(byte gameState, int player)`:
 - Genera un array de bytes amb informació addicional segons l'estat (de preparació, jugant o finalitzat), combinant dades del jugador i el seu oponent.

7. Gestió global de la partida (Game)

- `gameAlreadyCreated()`: Retorna `true` si la partida ja té almenys un jugador, indicant que ja s'ha iniciat.
- `getGameId()` i `getGameState()`:
 - `getGameId` retorna el codi únic de la partida.
 - `getGameState` retorna el codi que representa l'estat actual del joc.
- `leaveGame(int playerId)`: Permet que un jugador surti de la partida, eliminant-lo de la seva llista.
- `endGame()`: Finalitza la partida:
 - Canvia l'estat del joc a `FINISHED`.
 - Notifica a tots els jugadors l'estat final.
 - Elimina els jugadors passant per `removePlayer`.

2.3

GAMEBOARD

1. Constructor i inicialització

- `GameBoard(byte W, byte H)`
 - Inicialitza el tauler amb l'amplada (W) i l'altura (H).
 - Crea un array de bytes `board` amb mida $W * H$.
 - Clona el tauler original a `originalBoard` per poder comparar l'estat dels vaixells.

2. Recuperació de l'estat del tauler

- `getBoard()`: Retorna l'array de bytes que representa l'estat actual del tauler.
- `getBoardDimensions()`: Retorna un array de bytes amb les dimensions: amplada, altura i el

nombre total de cel·les ($W \cdot H$).

3. Mètodes d'utilitat i de validació

- `getIndex(int row, int col)`: Calcula l'índex a l'arranjament board per a una determinada fila i columna. Ja que implementem el board com a llista de byte i no una llista de llistes.
- `isValidCell(int row, int col)`: Comprova si les coordenades (fila i columna) són dins els límits del tauler.
- `isValidType(int t)`: Comprova si el tipus de vaixell és vàlid (entre 1 i 5).
- `isValidInstance(int n)`: Verifica que el número d'instància estigui entre 0 i 25 tal i com diuen les instruccions.
- `decodeCell(byte value)`: Decodifica un valor encriptat de la cel·la en un array d'enters: la posició 0 és el número d'instància i la posició 1 és el tipus.
- `cellValue(int row, int col)`: Retorna un array d'enters amb la instància i el tipus d'una cel·la determinada.
- `setCell(int row, int col, int n, int t)`: Si les coordenades, instància i tipus són vàlids, codifica la informació i l'emmagatzema tant a board com a originalBoard. Aquesta segona és útil per saber si un vaixell està enfonsat. Aquesta idea va néixer arran d'un comentari a l'enunciat del board.
- `encodeCell(int n, int t)`: Codifica el número d'instància i el tipus en un sol byte.
- `setSpecialCell(int row, int col, int state)`: Estableix un estat especial a una cel·la (0, per defecte; 1, error; 2, encert; 3, enfonsat), modificant el valor de la cel·la.
- `encodeSpecialState(int state)`: Codifica un estat especial en un byte.

4. Gestió dels vaixells i dispars

- `addVessel(int ri, int ci, int rf, int cf, int type, int instance)`
 - Intenta afegir un vaixell al tauler.
 - Valida el tipus i la instància.
 - Comprova que les coordenades estiguin dins del tauler i que el vaixell tingui la mida esperada (basada en un array de mides predefinit a l'enunciat).
 - També verifica que no hi hagi cap superposició amb altres vaixells.
 - Si tot és correcte, afegeix el vaixell a través del mètode `setCell` per cada cel·la ocupada.
- `shot(int r, int c)`
 - Processa un dispar en una cel·la especificada.
 - Si la cel·la no és vàlida s'executa una gestió d'error (retorna -1).
 - Si la cel·la està buida, marca la cel·la com a error (miss) i retorna 0.
 - Si la cel·la conté un vaixell, marca la cel·la com a encert (hit) i comprova si el vaixell ha estat

enfonsat:

- Si sí, marca tot el vaixell com a enfonsat i retorna 2.
- Si no, retorna 1.
- `markVesselAsSunk(int instance, int type)`
 - Recorre tot l'array board i per cada cel·la que coincideixi amb la instància i el tipus, marca la cel·la amb un estat de vaixell enfonsat.
- `isVesselSunk(int instance, int type)`
 - Comprova per tot el `originalBoard` si totes les cel·les corresponents a la combinació d'instància i tipus han estat tocades (marcades amb estat encertat).
 - Retorna `true` si el vaixell està enfonsat, `false` en cas contrari.

2.4

MESSAGES

Comentarem per sobre la dinàmica amb aquest conjunt de classes, perquè redactar-les individualment augmentaria exponencialment la mida d'aquesta memòria. Per fer-ho, parlarem de la classe base, `Message`, i de la més «complicada». Donarem per fet que la resta són una simplificació de la mateixa.

D'una banda, la classe `Message.java`. La classe `Message` és abstracta i representa un missatge genèric dins del protocol de comunicació. Tots els missatges del sistema estan caracteritzats per un tipus, definit per l'enum `MessageType`. Aquesta classe defineix l'estructura bàsica que haurà de seguir tot tipus de missatge del sistema.

1. **Atributs:**

- `private final MessageType type;`
 - Defineix el tipus del missatge.
 - Aquest atribut es defineix com a final, assegurant que el seu valor no es pugui modificar després de la construcció de l'objecte.

2. **Mètodes:**

- `Message(MessageType type)`
 - Constructor de la classe.
 - Inicialitza el missatge assignant el tipus rebut com a paràmetre.
- `getType()`
 - Retorna el tipus del missatge.
 - Permet accedir al valor de l'atribut `type` de forma segura.

D'altra banda, la classe `GameStatusMessage.java`. La classe `GameStatusMessage` representa un missatge d'estat de joc dins del protocol de comunicació. Aquest missatge transmet l'estat actual del joc, la mida del

tauler, l'estat de dos taulers i informació addicional. Hereta de la classe abstracta `Message` i, per tant, el seu tipus és fixat a `MessageType.GAMESTATUS`.

1. Atributs:

- `private byte gameState`: Emmagatzema l'estat actual del joc com a valor byte, que s'ha de correspondre amb un valor vàlid de l'enum `GameState`.
- `private int boardSize`: Indica la mida total del tauler.
- `private byte[] board1`: Array que representa l'estat del primer tauler.
- `private byte[] board2`: Array que representa l'estat del segon tauler.
- `private byte[] info`: Conté informació addicional relacionada amb l'estat del joc.

2. Mètodes:

- `GameStatusMessage(byte gameState, int boardSize, byte[] board1, byte[] board2, byte[] info)`
 - Constructor que inicialitza el missatge amb els paràmetres especificats.
 - Comprova que el valor byte passat com a `gameState` correspongui a un valor vàlid de `GameState`; si no, llença una `BattleshipException` amb `ErrorType.INVALID_STATE`.
 - Crida al constructor de la superclasse `Message` passant `MessageType.GAMESTATUS`.
- `getGameState()`: Retorna l'estat del joc com a byte.
- `getBoardSize()`: Retorna la mida total del tauler.
- `getBoard1()`: Retorna l'array de bytes que representa l'estat del primer tauler.
- `getBoard2()`: Retorna l'array de bytes que representa l'estat del segon tauler.
- `getInfo()`: Retorna l'array de bytes que conté informació addicional del missatge.
- `equals(Object obj)`
 - Compara aquest missatge amb un altre objecte.
 - Dos `GameStatusMessage` es consideren iguals si el seu tipus és `GAMESTATUS` i tots els camps (`gameState`, `boardSize`, `board1`, `board2` i `info`) són iguals.

3

SERVER

3.1

GAMEHANDLER

La classe `GameHandler` s'encarrega de gestionar la lògica del joc des del costat del servidor. Aquesta classe:

- Administra la comunicació amb el client.
- Processa els missatges rebuts i executa les ordres corresponents.
- Controla l'estat del joc i notifica als jugadors els canvis pertinents.

D'altra banda, tenim aquests mètodes:

1. `run()`: Aquest mètode és el cor de l'execució de la classe i hereta de `Thread`, pel que és important sobreescriure'l. Implementa un bucle principal on:
 - Es llegeixen missatges enviats pel client.
 - Es processa el missatge segons el seu tipus (per exemple: `CREATE`, `JOIN`, `SHOT`, etc.).
 - Es comprova l'estat actual del joc per validar si l'acció és permesa.
 - Es gestiona l'enviament de respostes i notificacions als clients.
2. `invalidStateMessage()`: Aquest mètode (hi ha dues variants) s'encarrega de:
 - Crear i enviar missatges d'error al client quan es rep un missatge invàlid segons l'estat actual del joc.
 - Les dues variants permeten tractar errors segons si es vol mostrar informació addicional (com el `playerId` i `gameId`).
3. `createCommand(CreateMessage message)`: Gestió de la creació d'un joc, assignació d'identificadors per al joc i jugador, i configuració inicial.
4. `joinCommand(String playerName)`: Processa la petició d'entrada d'un jugador en un joc existent.
5. `getConfigCommand()`: Recupera la configuració actual del joc (dimensions del tauler i configuració de vaixells).
6. `addVesselCommand(AddVesselMessage message)`: Afegeix un vaixell al tauler d'un jugador i actualitza l'estat del joc.
7. `shotCommand(ShotMessage message)`: Processa un cop disparat, determina contra qui va dirigit, i actualitza l'estat del joc segons si s'ha encertat, fallat o enfonsat un vaixell.
8. `handleShot(int shotResult, int playerId)`: Tracta el resultat d'un cop, enviant el missatge corresponent (`HIT` o `FAIL`) i canviant l'actiu si el cop ha fallat.
9. `getStatusCommand(GetStatusMessage message)`: Gestiona la petició d'estat del joc per un jugador.
10. `leaveCommand(LeaveMessage message)`: Permet que un jugador abandoni el joc i actualitza l'estat resultant.

El professorat no ens va proporcionar un codi correcte, en el sentit que no sobreescrivia els mètodes propis de la classe `Thread`. Això vol dir que, a efectes pràctics, en crear una nova instància d'aquesta classe a la classe `Server`, no es creava cap thread real. Encara que això no afectava gaire la implementació del single-player (o, si es vol, multiplayer en un mateix terminal), feia impossible implementar un sistema multiplayer complet (en diversos terminals).

3.2

SERVER

Aquesta classe la va fer el professorat, així que no entrarem en molt de detall. Simplement fer un parell de comentaris:

1. És important remarcar que a la implementació facilitada els atributs de Socket com `BattleshipComUtils` són específics del servidor, però això no hauria de ser així.
2. Aquestes classes es poden re-declarar en un entorn **multiplayer** per gestionar la comunicació entre múltiples clients, sense tenir una propietat exclusiva per al servidor en forma d'atribut. Això és perquè el Servidor haurà de crear un Socket i un `BattleshipComUtils` per a cada connexió establerta.

4

CLIENT

4.1

GAMECLIENT

La classe `GameClient` encapsula la lògica del joc. Segueix una seqüència d'estats segons el protocol establert i gestiona la comunicació amb el servidor mitjançant diferents missatges. Els atributs principals són:

- **client**: Instància per gestionar la connexió al servidor.
- **gameState**: Estat actual del joc (tipificat amb l'enum `GameState`).
- **gameId**: Identificador del joc.
- **playerId**: Identificador del jugador.

Els mètodes més importants són:

- `GameClient(Client client)`: Constructor que inicialitza el client i crida al mètode `init()`.
- `init()`: Inicialitza el joc i crida al mètode `run()`. Notem que no té res a veure amb el mètode `run()` del `GameHandler`, ja que aquest sobreescrirà el de la classe `Thread`.
- `run()`: Execució principal amb un bucle que gestiona la recepció de missatges, el processament segons el tipus i l'enviament de comandes.
- `readTerminal()`: Llegeix l'entrada del terminal per processar comandes de l'usuari.
- `arrayMessages(byte[] array)`: Converteix un array de bytes a una cadena separada per espais.
- `setCredentials(int playerId, int gameId)`: Assigna el valor a `playerId` i `gameId` si encara no estan assignats.
- `gameStatusMessage(GameStatusMessage message)`: Processa el missatge d'estat del joc, actualitzant l'estat intern i mostrant informació dels taulers.
- `help()`: Mostra per consola les comandes disponibles.
- Mètodes per enviar missatges al servidor: `createCommand()`, `joinCommand()`, `getConfigCommand()`, `addVesselCommand()`, `shotCommand()`, `getStatusCommand()` i `leaveCommand()`.

Sobre el mètode `run()` cal afegir el següent:

- Escolta constantment missatges enviats des del servidor.
- Processa els missatges rebuts segons el seu tipus.
- Gestió dels temps d'espera mitjançant `SocketTimeoutException`.

Quan el client intenta llegir un missatge amb `readMessage()`, si no arriba cap missatge en el temps estipulat, es llança una `SocketTimeoutException`. Aquest comportament fa que:

- Segons l'estat del joc (`gameState`), es mostri un missatge corresponent per consola. Per exemple:
 - `WAITING_PLAYERS`: mostra "Waiting for players..."
 - `SETUP`: mostra "Setting up the game..."
 - `PLAYING`: mostra "Playing the game..."
- Es cridi al mètode `readTerminal()` per permetre que l'usuari introdueixi una comanda manual.
- Si l'estat és `FINISHED`, es tanca la connexió amb el servidor.

ALTRES QÜESTIONS

5.1 MULTIPLAYER

Sí que hem implementat la funcionalitat de joc multijugador però l'hem desenvolupada en una branca separada anomenada `multiplayer`, amb l'objectiu de mantenir claredat i separació respecte a la branca principal.

Aquesta decisió es va prendre perquè el desenvolupament del `multiplayer` s'ha dut a terme al llarg d'aquestes últimes setmanes i, per tant, no vam arribar a la sessió de testing del 26 de març amb la funcionalitat completament acabada. Com que no vam poder realitzar proves amb múltiples clients i/o servidors vam considerar més adient deixar aquest desenvolupament en una branca independent.

Tanmateix, vam conservar tot allò que ja teníem funcional en aquesta branca abans de la sessió de proves. Concretament, es tracta dels següents punts

1. Lectura del paràmetre `-multiplayer` des de la línia de comandes.
2. Enviament d'aquesta flag a la classe `GameHandler`.
3. Validació que, en mode multijugador, no es creï cap partida amb bots.

La funcionalitat principal que diferencia la branca `multiplayer` de la branca `main` és la gestió de la creació de partides d'acord amb el que estableix l'enunciat. Per tal d'implementar aquest comportament hem creat una classe estàtica anomenada `GameManager`, que conté una cua FIFO on s'emmagatzemen les partides pendents d'assignació. Quan es crida al mètode `createCommand()` des de la classe `GameHandler` s'invoca la funció corresponent de `GameManager` per afegir una nova partida a la cua. De la mateixa manera, en produir-se un `join` es recupera la primera partida disponible utilitzant les operacions `put()` i `poll()`, respectivament.

Un dels principals problemes que vam trobar i que ja ha estat comentat anteriorment a la secció 3.1, va ser que el mètode `start()` de la classe `GameHandler` no estava sobreescrivint correctament el mètode de la classe `Thread`. Com a conseqüència, els fils no es creaven adequadament. Un cop identificat aquest error la implementació va quedar gairebé completada.

D'altra banda també vam tenir dificultats amb la funció `notifyStatus()` de la classe **BattleshipGame**. Aquesta funció s'encarrega de notificar l'estat del joc als diferents jugadors i per fer-ho necessita un objecte `BattleshipComUtils` per tal d'enviar la informació a través del socket. Com que no podíem modificar la signatura de la funció, opció que hauria simplificat molt el procés, vam optar per crear un diccionari on cada jugador tingués associat el seu socket corresponent. D'aquesta manera la classe pot identificar quin socket ha d'utilitzar per enviar la informació a cada jugador.